

RAPPORT DE PROJET

Master Informatique - Structures de données avancées

Route planning

Répondre vite à des requêtes de plus court chemin

Réalisé par : Mathura SANTHALINGAM
Encadrant : M. Bodini

Sommaire

Introduction.....	2
1. Extraction et pré-traitement des données.....	2
1.1 Le choix du format PBF.....	2
1.2 Pipeline de traitement avec Pyosmium.....	2
1.3 Calcul des poids (formule de Haversine).....	3
2. Le format CSR (Compressed Sparse Row).....	3
2.1 Les limites de l'approche naïve (listes chaînées).....	3
2.2 La solution : Compressed Sparse Row (CSR).....	4
2.3 Algorithme de construction en C.....	4
3. Dijkstra et min-heap : la baseline.....	4
3.1 Le tas binaire.....	4
3.2 L'optimisation paresseuse.....	5
4. L'algorithme A* : la recherche orientée.....	5
4.1 La fonction d'évaluation $f(v)$	5
4.2 L'heuristique admissible - Haversine.....	5
4.3 Adaptation technique.....	6
5. L'algorithme ALT : accélération.....	6
5.1 L'inégalité triangulaire.....	6
5.2 Les deux phases de l'algorithme.....	6
6. Contraction Hierarchies (CH) : le paroxysme de l'optimisation.....	7
6.1 Pré-traitement et ordre de contraction.....	7
6.2 Recherche témoin et graphe ascendant.....	8
7. Évaluation, benchmarks et analyse des résultats.....	8
7.1 Données brutes de l'expérience.....	8
7.2 Données brutes.....	9
7.3 Analyse de l'espace de recherche : extractions et relaxations.....	9
7.4 Analyse du temps de réponse.....	11
7.5 Analyse de la mémoire.....	12
Conclusion.....	13

Introduction

Ce rapport a été réalisé dans le cadre du cours de Structures des Données Avancées à l'Université Sorbonne Paris Nord (USPN). L'objectif de ce travail est de concevoir et d'évaluer plusieurs méthodes d'accélération algorithmiques.

Le calcul du plus court chemin sur des réseaux routiers à grande échelle est un problème d'algorithmique, en effet, sur un réseau dense comme celui de l'Île-de-France avec plusieurs millions d'intersections et de segments routiers, les implémentations de l'algorithme de Dijkstra font face à des limites de temps d'exécution et de consommation mémoire.

Ce projet a pour but de concevoir un moteur de Route Planning optimisé en langage C. Ce rapport détaille le pipeline de traitement des données, la modélisation mathématique du graphe en mémoire, et l'évolution de notre moteur de Dijkstra vers des heuristiques avancées (A* et ALT).

1. Extraction et pré-traitement des données

Les données géographiques brutes utilisées pour ce projet proviennent d'extractions OpenStreetMap (OSM) via Geofabrik. Pour des raisons de simplicité, le parsing de ces données n'a pas été réalisé en C, mais via un pipeline hybride en Python.

1.1 Le choix du format PBF

Le format classique d'OSM c'est le xml ([.osm](#)), mais comme le xml est syntaxiquement très lourd et consomme beaucoup de mémoire lors du parsing. Nous avons choisi le format binaire Protocolbuffer Binary Format ([.osm.pbf](#)). Ce format compressé permet de manipuler des zones denses sans saturer la mémoire vive (RAM).

1.2 Pipeline de traitement avec Pyosmium

Le script d'extraction est basé sur la bibliothèque [pyosmium](#) en utilisant le design pattern [SimpleHandler](#) déjà existant. Cette approche permet une lecture en streaming du fichier binaire, ne déclenchant des fonctions que s'il y a rencontre d'entités géographiques ciblées.

On a choisi d'optimiser ce pipeline et l'adapter plus à notre contexte :

- ❖ **Stockage $O(1)$** : Les nœuds temporaires sont stockés dans un dictionnaire Python, permettant une récupération instantanée des coordonnées.
- ❖ **Filtrage typologique** : Seules les routes pertinentes (motorway, trunk, etc.) sont conservées, on élimine donc les bâtiments, sentiers, etc.
- ❖ **Re-mappage des identifiants** : Les identifiants OSM étant très grands et non continus, on les a re-mappés de 0 à N-1 pour utiliser ces identifiants comme indices de tableaux en C.

1.3 Calcul des poids (formule de Haversine)

Les données OSM donnent la topologie (qui est connecté à qui) et la géométrie (les coordonnées GPS), mais pas la longueur physique des segments. On a donc utilisé la **formule de Haversine** pour le poids de chaque arête :

$$d = 2R * \arcsin(\sqrt{a}) = 2R * \arctan\left(\frac{\sqrt{a}}{\sqrt{1-a}}\right)$$
$$a = \sin\left(\frac{lat2 - lat1}{2}\right)^2 + \cos(lat1) * \cos(lat2) * \sin\left(\frac{lon2 - lon1}{2}\right)^2$$

Avec R le rayon de la Terre, et a est le carré de la moitié de la distance cordale entre les points.

L'utilisation de la fonction arc-tangente est nécessaire pour éviter des erreurs de flottants, en effet, parfois a tendance à dépasser un peu 1.0, provoquant un plantage de la fonction arc-sinus traditionnelle.

Les fichiers finaux fournis au moteur C sont **noeuds.csv** (identifiants et coordonnées) et **arete.csv** (source, cible et distance calculée).

2. Le format CSR (Compressed Sparse Row)

Sur un réseau de plusieurs millions d'arêtes, la manière dont le graphe est modélisé dans la mémoire vive de l'ordinateur influe directement les performances globales du système.

2.1 Les limites de l'approche naïve (listes chaînées)

La méthode classique consiste à utiliser un tableau de pointeurs, où chaque case pointe vers une liste chaînée représentant les voisins d'un nœud. Cette approche présente deux défauts critiques pour le Route Planning :

- ❖ **Surconsommation de mémoire** : Chaque voisin nécessite l'allocation d'un nœud de liste contenant un pointeur next (8 octets sur une architecture 64 bits), ce qui fait monter la consommation de mémoire.
- ❖ **Localité spatiale** : Les nœuds alloués dynamiquement par de nombreux appels à malloc sont dispersés aléatoirement dans la RAM. Lors du parcours des voisins, le processeur ne peut pas anticiper les adresses mémoire d'où "cache misses", l'obligeant à attendre pour récupérer la donnée en RAM au lieu de la lire directement dans sa mémoire cache.

2.2 La solution : Compressed Sparse Row (CSR)

Pour pallier ces limites, nous utilisons la structure **CSR**, qui aplatit le graphe dans deux uniques tableaux contigus :

- ❖ **arete** : Un tableau contenant **toutes** les arêtes du graphe (cible et poids), rangées nœud par nœud. Le nœud source n'est plus stocké, ce qui représente une énorme économie de mémoire par rapport à un tableau classique de triplets (**u, v, w**).
- ❖ **first_arete** : Un tableau d'offsets (indices) de taille **V+1**. Il indique à quel indice du tableau **aretes** commencent les voisins d'un nœud donné. Les voisins d'un nœud **U** se trouvent strictement entre les indices **first_arete[U]** et **first_arete[U+1]**.

2.3 Algorithme de construction en C

La contrainte des tableaux contigus est qu'ils ne peuvent pas être redimensionnés. On a donc fait le chargement en 3 lectures sur le fichier **aretes.csv** :

- ❖ **1ère lecture - découverte** : lecture pour trouver l'id maximal (nombre de nœuds **V**) et le nombre total d'arêtes **E**, pour allouer la mémoire de manière exacte.
- ❖ **2ème lecture - calcul des degrés** : on compte le nombre de voisins sortants pour chaque nœud pour faire une somme cumulative sur ces degrés et générer le tableau **first_arete**.
- ❖ **3ème lecture - remplissage (routing)** : dernière lecture à l'aide d'un tableau d'offsets temporaires (**current_offset**). Chaque arête lue est insérée à sa position définitive en mémoire.

3. Dijkstra et min-heap : la baseline

L'algorithme de Dijkstra explore le graphe de manière concentrique à partir du point de départ. Son efficacité dépend entièrement de la structure de données utilisée pour gérer la file de priorité.

3.1 Le tas binaire

Nous avons modélisé un **tas binaire (ordre min)** sous forme de tableau plat ainsi toutes les données de la file de priorité sont rangées de manière contiguë et on évite de faire des centaines de milliers de **malloc()** inutiles.

Pour un élément situé à l'indice **i** :

- ❖ Son parent est à l'indice $\lfloor (i - 1) / 2 \rfloor$.
- ❖ Ses enfants sont aux indices $2i + 1$ et $2i + 2$.

Le tas utilise des structures **element_tas_t** contenant l'identifiant du sommet et la distance cumulée. C'est cette distance qui sert de **clé de tri** pour les opérations de remontée et de descente en $O(\log N)$.

3.2 L'optimisation paresseuse

Quand un chemin plus court est trouvé vers un sommet déjà présent dans le tas, l'algorithme doit modifier la valeur existante (par diminution de clé). Mais retrouver un nœud arbitraire dans un tas prend un temps linéaire $O(V)$.

Pour résoudre ce problème, on s'est inspirés de la **stratégie paresseuse vue en cours pour les tas de Fibonacci**. Dans un tas de Fibonacci, on ne cherche pas à avoir une structure parfaite à chaque modification, on reporte le travail coûteux au moment de l'extraction. Nous avons appliqué le même principe à notre Dijkstra :

- ❖ **Duplication** : quand une meilleure distance est découverte, on insère simplement une nouvelle paire dans le tas, sans chercher à supprimer l'ancienne.
 - ❖ **Suppression paresseuse** : l'ordre min du tas garantit que la paire avec la plus petite distance (la plus à jour) sortira en premier. Donc quand l'ancienne paire, devenue obsolète, est extraite plus tard, on l'ignore si son coût est supérieur à la meilleure distance connue.
-

4. L'algorithme A* : la recherche orientée

Bien que Dijkstra soit optimal, son exploration dans toutes les directions lui fait perdre beaucoup de temps à explorer des routes menant à l'opposé de la cible. L'algorithme A* corrige ce défaut en orientant géographiquement la recherche.

4.1 La fonction d'évaluation $f(v)$

Dans A*, la file de priorité ne trie plus les nœuds selon la distance parcourue, mais selon un score global estimé $f(v)$:

$$f(v) = g(v) + h(v)$$

- ❖ $g(v)$: coût réel validé depuis le départ.
- ❖ $h(v)$: heuristique, la distance estimée restante jusqu'à l'arrivée.

4.2 L'heuristique admissible - Haversine

Pour que l'algorithme A* garantisse l'optimalité du chemin, l'heuristique $h(v)$ doit être **admissible**, c'est-à-dire qu'elle ne doit **jamais surestimer la distance réelle**.

La ligne droite étant le chemin le plus court entre deux points, la distance à vol d'oiseau calculée par la formule de Haversine (via les coordonnées chargées depuis **noeuds.csv**) constitue une heuristique admissible pour les réseaux routiers.

4.3 Adaptation technique

L'implémentation de A* a nécessité d'ajuster notre tas binaire :

- ❖ **Clé de tri** : les remontées et descentes dans l'arbre se basent que sur $f(v)$.
- ❖ **Vérification de la lazy deletion** : la structure stocke la vraie distance $g(v)$ en plus du score $f(v)$. Au moment d'extraire un nœud du tas, on vérifie s'il est obsolète en comparant son coût réel (**courant.g > distances[u]**). C'est indispensable, car le coût g est la seule métrique qui représente une véritable distance physique.

Orienter la recherche divise de manière importante le nombre de nœuds extraits (et donc le temps de calcul), en restreignant l'espace d'exploration autour du segment Départ-Arrivée.

5. L'algorithme ALT : accélération

L'heuristique de la ligne droite (Haversine) a des limites face à certains obstacles comme les fleuves, chaînes de montagnes, absence de ponts, etc. Dans ces cas de figure, l'estimation est mauvaise, et A* devient un parcours proche de celui de Dijkstra.

L'algorithme ALT (A*, Landmarks, Triangle inequality) génère une heuristique basée sur la topologie du graphe, le tout sans recourir aux coordonnées GPS.

5.1 L'inégalité triangulaire

Dans un triangle formé par un nœud **U**, l'arrivée **V**, et un point de repère fixe sur la carte appelée Landmark (**L**), la distance directe est toujours supérieure ou égale à la différence des détours :

$$dist(U, V) \geq |dist(U, L) - dist(V, L)|$$

Puisque cette soustraction ne surestime jamais la véritable distance routière, elle constitue une heuristique admissible parfaite.

5.2 Les deux phases de l'algorithme

L'algorithme ALT est une illustration concrète du compromis espace-temps en informatique :

1. Phase de pré-calcul : Avant de traiter la moindre requête, le système sélectionne K nœuds aléatoires (les Landmarks). Pour chacun d'eux, un algorithme de Dijkstra est exécuté pour calculer les distances exactes vers l'ensemble de la carte (données sont stockées en RAM).

2. Phase de requête : ALT fonctionne un peu comme A*, mais le calcul de la ligne droite est remplacé par une interrogation en temps constant des tableaux de pré-calculs. Pour augmenter la précision, l'algorithme calcule l'inégalité pour les K Landmarks et conserve la valeur maximale trouvée. L'espace de recherche est donc diminué.

On a fixé $K=10$ landmarks sélectionnés aléatoirement, ce qui en fait une limite : une sélection géographique aux bords de la région fournirait de meilleures heuristiques et réduirait sans doute davantage l'espace de recherche surtout sur les trajets dont le chemin optimal longe un obstacle naturel comme un fleuve.

6. Contraction Hierarchies (CH) : le paroxysme de l'optimisation

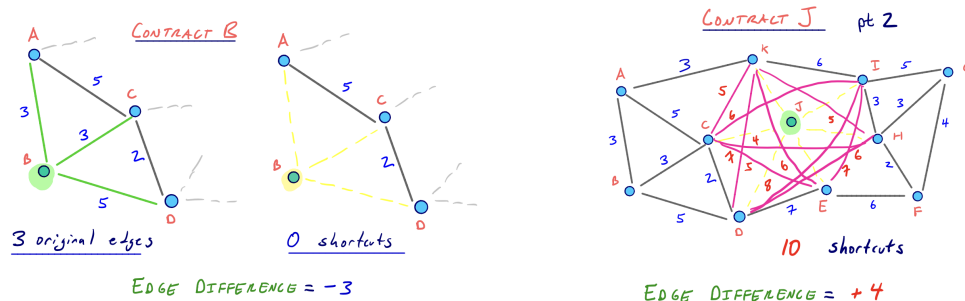
On a aussi implémenté l'algorithme des Contraction Hierarchies (CH). Cette méthode repose sur une phase de pré-traitement très lourde et une phase de requête très rapide.

Il y a un vrai parallèle avec la **compression de chemins vue en cours dans la structure Union-Find**. Dans les deux cas, on hiérarchise pour optimiser et rendre toutes les requêtes futures presque instantanées.

6.1 Pré-traitement et ordre de contraction

L'efficacité de CH dépend de l'ordre dans lequel les sommets sont contractés. Au début nous avons implémenté pour un ordre naïf de 0 à N qui s'est avéré très long donc on s'est rabattu sur l'heuristique étudiée par **John Lazarsfeld**, basée sur l'**Edge Difference** :

$$\text{Edge Difference} = (\text{Nombre raccourcis ajoutés}) - (\text{Nombre d'arêtes supprimées})$$



Figures extraites de l'étude de John Lazarsfeld

L'exemple montre que contracter un nœud périphérique comme **B** est très rentable : on supprime 3 arêtes sans ajouter de raccourci (score de -3), ce qui allège le graphe. À l'inverse, contracter trop tôt un carrefour comme **J** oblige à créer 10 raccourcis pour seulement 6 arêtes supprimées (score de +4), ce qui densifie inutilement le réseau. Trier par Edge Difference croissante garantit donc de nettoyer les zones simples (B) et repousser les intersections complexes (J) à la fin pour éviter l'explosion de la mémoire.

Pour maintenir cet ordre à jour malgré les contractions successives qui modifient la topologie, on utilise à nouveau une **stratégie de mise à jour paresseuse (lazy update)** avec une file de priorité, évitant un re-calcul global trop coûteux.

6.2 Recherche témoin et graphe ascendant

Quand un sommet est contracté, il faut garder les plus courts chemins qui le traversent. Nous lançons donc un Dijkstra local entre ses voisins. Si aucun chemin alternatif satisfaisant n'est trouvé, un raccourci virtuel est inséré. Une fois la contraction terminée, on filtre le réseau pour ne garder que le **graphe ascendant** où l'on ne peut voyager que vers des nœuds de rang supérieur. Cette stratégie revient à ce qu'on a fait en cours avec **Union-Find** : dans les deux cas, la recherche se fait par une **hiérarchie** (le rang) et que vers le haut. En effet, la recherche part simultanément de la source et de la destination pour grimper vers les sommets de la hiérarchie au sein du seul graphe ascendant. Ces deux explorations se rejoignent sur un nœud de rang supérieur. Cette recherche bidirectionnelle limitée à ce graphe réduit permet de restreindre l'espace d'exploration et rend le calcul presque instantané.

Petite précision : notre graphe est non-orienté donc les deux recherches utilisent le même graphe ascendant et parcourent toutes les deux les arêtes montantes (contrairement à un Dijkstra bidirectionnel classique). Cette propriété reste bonne par la symétrie des poids qui garantit que les arêtes montantes depuis la destination couvrent les mêmes chemins optimaux.

7. Évaluation, benchmarks et analyse des résultats

Pour valider l'efficacité de nos moteurs, on a mis en place une série d'évaluations inspirées de la méthodologie du **TP1 de SDA** (analyse amortie). Nous avons repris et adapté le fichier **analyzer.c** de ce TP pour mesurer le temps, l'espace de recherche et la RAM.

7.1 Données brutes de l'expérience

Pour avoir des résultats corrects et éviter que le hasard favorise une majorité de trajets très rapides ou très longs, nous avons implémenté le protocole dont il est question dans l'énoncé :

Reproductibilité : avec une graine fixe (**SEED_EVAL = 42**), pour que les mêmes points de départ et d'arrivée soient générés à chaque exécution pour comparer les algorithmes.

Contrôle des échantillons : on a défini un lot de **300 requêtes valides** réparties en trois catégories, les distances ont été sélectionnées empiriquement après plusieurs séries de tests pour avoir un bon équilibre :

- ❖ **Trajets courts** : distance réelle inférieure à 25 km.
- ❖ **Trajets moyens** : distance réelle comprise entre 25 km et 50 km.
- ❖ **Trajets longs** : distance réelle supérieure à 50 km.

Filtrage par rejet : Si le générateur tire une requête impossible (distance **INFINI**), ou appartenant à une catégorie déjà remplie, on ignore le résultat et une nouvelle paire est tirée.

Validation de correction : L'algorithme de Dijkstra est notre vérité terrain. Les distances calculées par A*, ALT et CH sont comparées à Dijkstra avec une tolérance petite pour absorber les petites erreurs d'accumulation flottante propres au processeur.

7.2 Données brutes

Bilan des prétraitements : ALT (10 Landmarks) 2.53 secondes contre 128.17 secondes pour Contraction Hierarchies.

Bilan des 300 requêtes :

Algorithmes	Temps Moyen (ms)	Extractions Moyennes	Relaxations Moyennes	Ecart-type Temps	RAM (Mo)
Dijkstra	106.8132	637303.9200	653370.4133	75.0977	92.96
A*	38.3350	131930	138247.3100	43.5582	92.96
ALT	12.5504	61254	64823.9933	17.6856	92.96
CH	0.5857	940	2106.6433	0.2722	177.52

On voit l'importance du **pré-traitement**. L'approche de Dijkstra (106,81 ms, 637 304 extractions) est largement surpassée par ALT (12,55 ms), et surtout par Contraction Hierarchies (CH) qui n'explore que 940 nœuds, CH répond en seulement 0,59 ms avec une stabilité absolue (écart-type de 0,27), même si cette vitesse s'obtient au prix d'une consommation mémoire importante (177,52 Mo contre 92,96 Mo pour les autres)

7.3 Analyse de l'espace de recherche : extractions et relaxations

Pour comprendre pourquoi certains algorithmes sont plus rapides, nous avons mesuré deux indicateurs clés qui définissent l'espace de recherche de nos algorithmes :

- ❖ **Les extractions :** Ça correspond au nombre de fois qu'un sommet est retiré de la file de priorité (tas binaire) et reflète le nombre de nœuds explorés par l'algorithme.

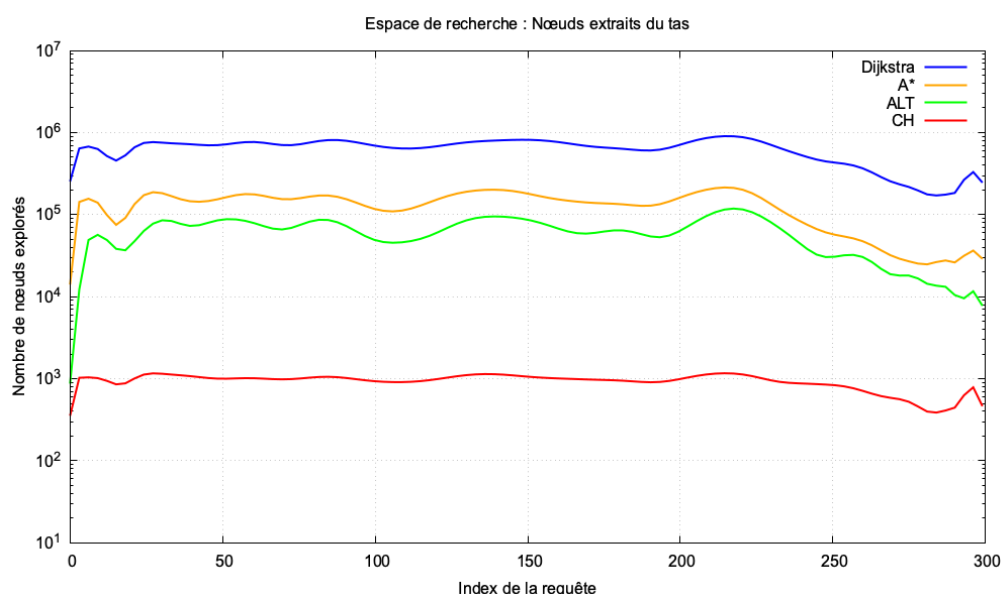


Figure 1 : Nombre de nœuds extraits du tas binaire par algorithme (échelle logarithmique).

- ❖ **Les relaxations** : Ça représente le nombre de fois où l'algorithme trouve un chemin plus court vers un voisin et met à jour sa distance dans le tableau des résultats. C'est l'indicateur principal du coût de calcul comme chaque relaxation utile nécessite d'ajouter ou de mettre à jour un élément dans le tas binaire (opération en $O(\log N)$).

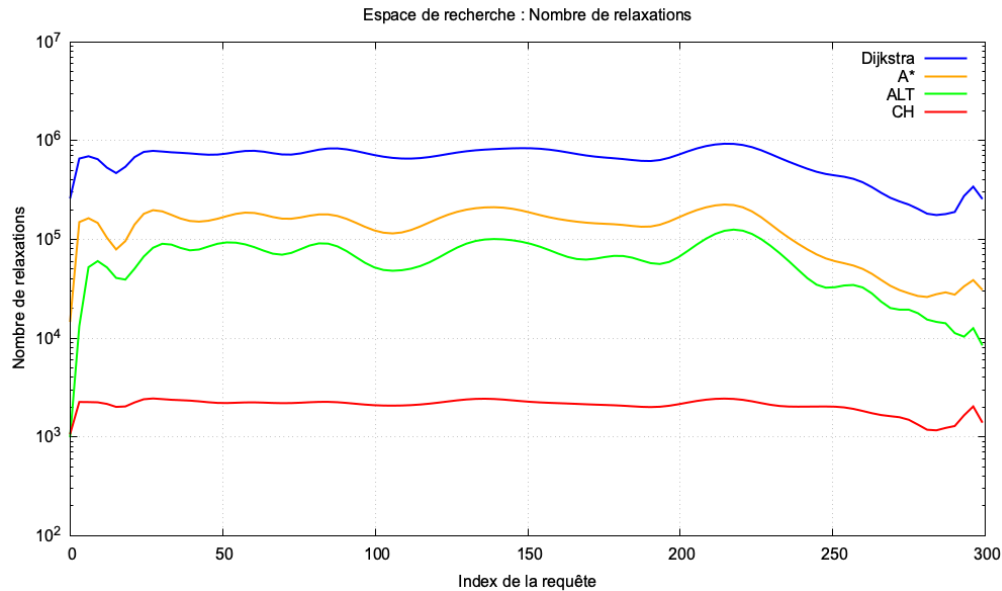


Figure 2 : Nombre de relaxations par algorithme (échelle logarithmique).

Ces métriques expliquent les écarts de performance et l'importance des heuristiques :

- **Dijkstra (bleu)** : explore aveuglément un tiers du réseau et sature la structure de relaxations.
- **A* et ALT (jaune et vert)** : divisent ces opérations grâce à leurs heuristiques
- **CH (rouge)** : atteint le paroxysme de l'optimisation comme ses "raccourcis" autoroutiers limitent la recherche à 940 nœuds et 2 100 relaxations, ignorant presque tout le reste du réseau.

La chute des courbes à la fin (index 200-300) s'explique par nos quotas : les trajets longs et moyens remplissent leur quota en premier, les dernières requêtes enregistrées sont donc que des trajets courts (< 25 km) qui sont donc plus rapides à calculer.

Tableau des résultats :

Algorithmes	Débit (req/s)	Moyenne (ms)	p50 / Médiane (ms)	p95 (ms)	Max (ms)
Dijkstra	9	106.81	128	243	270.22
A*	26	38.33	20.08	135.17	212.640
ALT	79	12.55	5.12	45.76	114.11
CH	1707	0.59	0.63	0.95	1.645

Pré-traitement :

Algo	Temps (secondes)	RAM (Mo)
ALT	2.81	115.97
CH	121.53	80.88

Ces tableaux illustrent le **gain de performance** apporté par le pré-traitement. L'approche de Dijkstra limite le système à 9 requêtes/seconde et se dégrade sur les longs trajets (le p95 monte à 230 ms). À l'inverse, Contraction Hierarchies (CH) fait monter le débit à 1707 req/s avec un temps moyen de 0,59 ms tout en ayant un stabilité de 95%, des requêtes et le pire cas absolu (max) qui est à 1,645 ms, montrant que le temps de réponse devient presque indépendant de la distance du trajet même si elle demande un temps de calcul initial bien plus important (plus de 2 minutes).

7.4 Analyse du temps de réponse

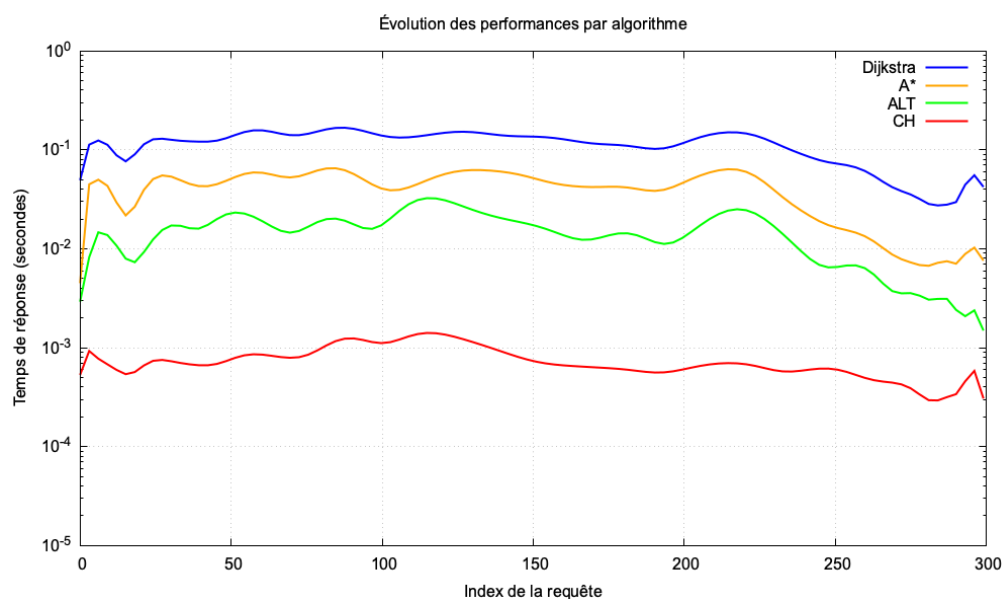


Figure 3 : Temps de réponse en secondes par requête (échelle logarithmique).

Comme prévu, la Figure 3 montre une corrélation entre le nombre de nœuds extraits et le temps d'exécution. Grâce à ses raccourcis bidirectionnels, CH s'exécute en **0.6 milliseconde(10^{-3})**, rendant le calcul presque instantané. Aussi, l'écart-type très faible de CH démontre une grande stabilité : la résolution est toujours immédiate, que le trajet fasse 2 ou 100 kilomètres.

7.5 Analyse de la mémoire

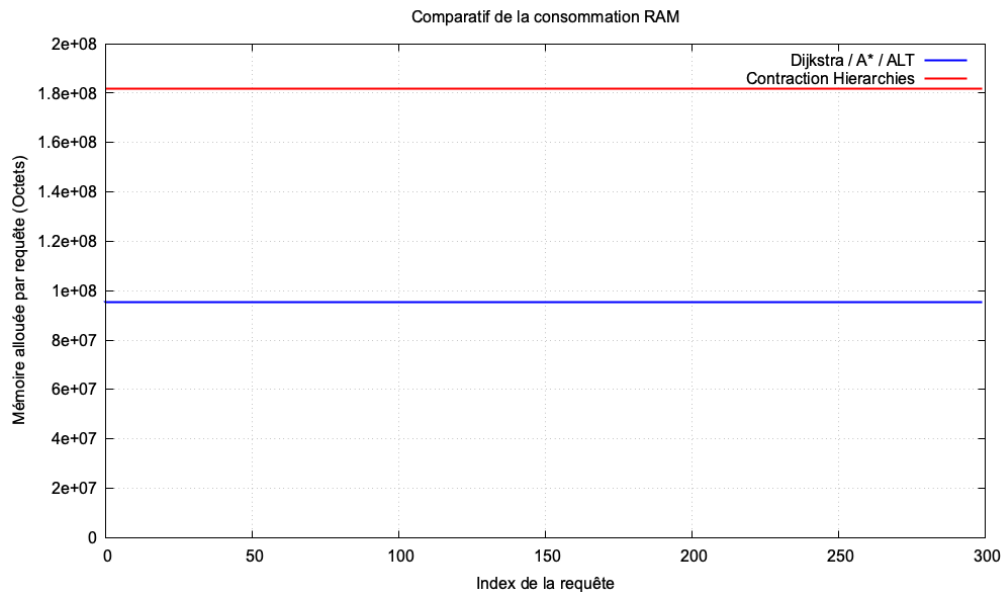


Figure 4 : Mémoire vive dynamique allouée lors d'une requête.

La Figure 4 et nos données de prétraitement montrent l'importance du compromis entre l'espace et le temps. La vitesse de Contraction Hierarchies a un coût :

- ❖ **En temps de compilation** : Il a fallu **128 secondes** de calculs lourds (recherches témoins et contractions) avant de pouvoir lancer la première requête.
- ❖ **En espace mémoire** : Le graphe ascendant et les raccourcis virtuels font monter l'empreinte mémoire du système (environ 177 Mo alloués contre 92 Mo pour les algorithmes classiques).

Stabilité selon la distance (Courtes vs Longues requêtes) :

L'analyse de nos centiles (p50 et p95) permet de répondre à la question de la stabilité des algorithmes face à l'allongement de la distance :

- ❖ Pour **Dijkstra et A*** : l'écart entre la médiane (p50) et le pire cas (p95) est conséquent (Dijkstra passe de 128 ms à 243 ms). Leurs performances se dégradent de manière au fur et à mesure que la cible s'éloigne, car le cercle d'exploration s'élargit.
- ❖ L'algorithme **ALT** a une amélioration avec une bonne médiane (5,12 ms) mais il reste sensible aux longues distances. Son p95 atteint 45.76 ms : cette variation s'explique par le fait que si le trajet demandé s'aligne mal avec les repères qu'on a pré-calculés, l'inégalité triangulaire perd en précision.
- ❖ **Contraction Hierarchies** démontre une stabilité absolue : son p50 et son p95 sont presque identiques car il n'emprunte que les routes pré-calculées du graphe ascendant.

Conclusion

Pour conclure, ce projet a été très enrichissant car il nous a permis de mesurer le décalage entre la théorie académique et la réalité pratique. On a pu constater par nous-mêmes qu'en matière de calcul de plus court chemin sur des graphes à grande échelle, comme le réseau routier d'Île-de-France, connaître et implémenter l'algorithme de Dijkstra n'est que la toute première étape.

En effet, quand on a un volume de données aussi massif, on a compris que l'utilisation du format CSR pour optimiser l'empreinte mémoire et la localité du cache, ainsi que l'implémentation de stratégies paresseuses (qui nous a permis de mettre en pratique l'idée de l'évaluation paresseuse vue en cours avec les tas de Fibonacci), se sont révélées être des choix d'architecture indispensables pour que notre programme soit réellement performant.

Surtout, le fait de coder et d'évaluer des algorithmes de plus en plus poussés (A*, ALT et enfin Contraction Hierarchies) nous a fait prendre conscience de toute la puissance du compromis entre l'espace et le temps consommés. Et comme on l'a étudié avec la technique de compression de chemins pour la structure Union-Find, nous avons appris qu'il est souvent stratégique de sacrifier de l'espace mémoire à travers un pré-traitement assez lourd pour pouvoir accélérer les recherches futures. On a donc obtenu un moteur capable de répondre aux exigences de vitesse presque instantanée du domaine du Route Planning.